OPEN ACCESS

Edited and reviewed by:
Research Division, Borel Sigma Inc.

Correspondence:
Raja Ram M
zi-us@borelsigma.com

Specialty section:
Actuarial Data Science & MLOps

Received: 04 August 2026
Accepted: 04 August 2026
Published: 04 August 2026

Citation:
Ram M R, Muskan S, Jain V and Swain K (2026) FEATSRV: A Leakage-Free Point-in-Time Feature Store for Actuarial Data Science. *Technical Report.* doi:10.5281/zenodo.21798383

Repository:
Public fork

Roles:
Kryptur - Raja R.: R&D
Data-T - Muskan S.: Infrastructure
AE - DCN
Zi-us - Quantum & AI

FEATSRV: A Leakage-Free Point-in-Time Feature Store for Actuarial Data Science

***Raja Ram M*[1,4]*, *Muskan S*[2], *Vipul Jain*[3], *Kalinga Swain*[3]**

[1] *Zius Quantum R&D Center (Zi-us: Quantum & AI)*
[2] *Data T Research Org (Data-T: Project Infrastructure)*
[3] *AE Quantum Research Division (AE: Digital & Cloud Networks)*
[4] *Kryptur OU / Krypur Quantum R&D*

Data Manager: Borel Sigma Data Center
[*] *Correspondence: zi-us@borelsigma.com*

Keywords: feature store, point-in-time join, data leakage, actuarial machine learning, Feast, Redis, FastAPI

Abstract

Data leakage is the predominant cause of failure in machine learning models deployed in insurance. We present **FEATSRV**, an open-source, end-to-end feature store that guarantees point-in-time correctness for actuarial feature serving. The system combines Feast, Redis, PySpark, and FastAPI to offer both offline batch training-set generation and online real-time feature lookups, secured via Cloudflare Tunnel. A companion terminal CLI and a browser-based dashboard enable actuaries and data scientists to interact with the feature store without writing SQL. All components are publicly available at GitHub, together with a ready-to-use synthetic insurance dataset (Nimbus) that exhibits realistic slowly-changing dimensions and temporal complexity. The archival DOI is 10.5281/zenodo.21798383.

1 Introduction

Insurance portfolios contain highly temporal data: policies are issued, addresses change, claims are reported, and liabilities develop over time. When training a predictive model (e.g., fraud scoring or loss reserving), it is essential to use only the information that would have been known at the moment of prediction - never future data. Violating this principle, known as **data leakage**, leads to over-optimistic validation metrics and catastrophic performance degradation in production [1].

Traditional approaches rely on ad-hoc SQL scripts or manual feature engineering pipelines, which are error-prone, slow, and difficult to audit. Feature stores have emerged as a disciplined alternative, encapsulating complex temporal logic behind a clean API [2]. In this work, we design, implement, and publicly release **FEATSRV**, a production-grade point-in-time feature store tailored to the actuarial domain.

doi: 10.5281/zenodo.21798383

2 Related Work

Feature stores have gained traction in the machine-learning operations (MLOps) community. Feast [2] is an open-source feature store that supports point-in-time joins and online serving via Redis. PySpark [3] provides scalable data processing for offline feature generation. FastAPI [4] is a modern Python web framework for building APIs. Cloudflare Tunnel [5] offers a zero-trust networking solution, hiding origin IPs and automatically providing HTTPS.

FEATSRV integrates these components into a unified, easy-to-deploy stack that is specifically designed for the actuarial use case, including a synthetic dataset generator that replicates real-world temporal patterns.

3 System Architecture

The system consists of five principal layers, summarised in Figure 1.

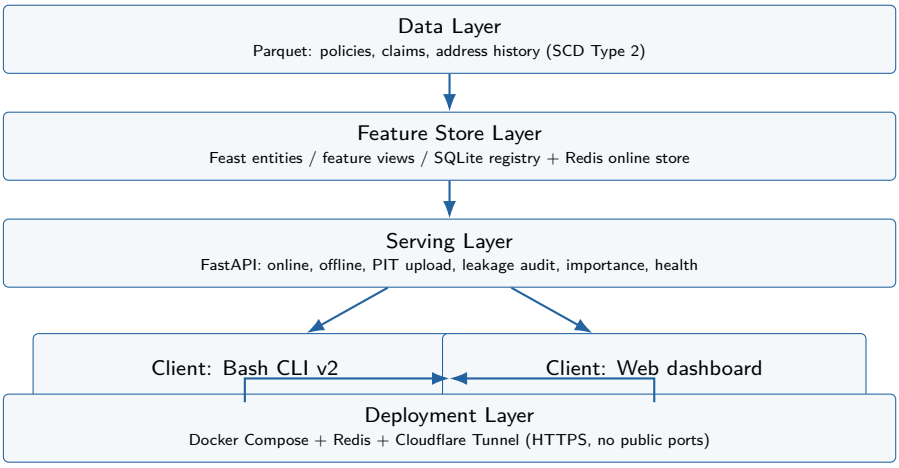


Figure 1: FEATSRV five-layer architecture: from temporal Parquet sources through Feast/Redis to FastAPI clients and secure deployment.

Data Layer
Synthetic insurance data (policies, claims, address history) stored as Parquet files. The data is designed with slowly-changing dimensions (SCD Type 2) to mimic real temporal challenges [7].

Feature Store Layer
Feast manages entities, feature views, and a SQLite registry. Offline features are read directly from Parquet; online features are materialised into Redis [6].

Serving Layer
A FastAPI application exposes REST endpoints for online feature lookups, batch offline training-set generation, file-based PIT joins, leakage audits, feature importance, and health checks.

Client Layer
A Bash-based interactive CLI (v2.0.0) provides seven numbered services, non-interactive flags, and automatic dashboard launching. A browser dashboard renders real data from static JSON exports and includes a print-to-PDF button.

doi: 10.5281/zenodo.21798383

Deployment Layer
Docker Compose orchestrates the API and Redis containers on a cloud virtual private server. Cloudflare Tunnel provides secure HTTPS without exposing any public ports.

4 Methodology

4.1 Point-in-Time Correctness

For a label timestamp t, every feature must be sourced from the most recent record with valid_from $\leq t <$ valid_to (or valid_to = NULL). Feast implements this automatically via its `get_historical_features` API, which performs a temporal as-of join across multiple feature views. We validated the correctness by querying addresses before and after a policyholder's move - the returned address matched the expected version exactly. Figure 2 illustrates the join semantics.

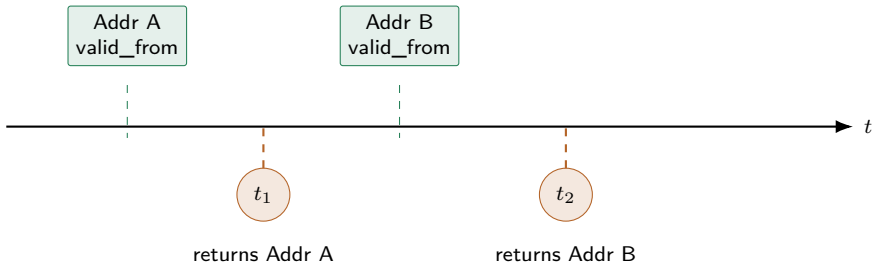


Figure 2: Point-in-time address join: prediction times t_1 and t_2 retrieve only historically valid address versions (no future leakage).

4.2 Offline Training Set Generation

Users upload a CSV containing entity IDs and an event timestamp. The API passes it to Feast, which returns a leakage-free DataFrame. For scalability, we also provide a PySpark script that performs the same point-in-time join using SQL, ready to run on GPU-accelerated compute clusters.

4.3 Online Feature Serving

The latest feature values are materialised from Parquet into Redis via `feast materialize`. The FastAPI endpoints `/online/policy/{id}` and `/online/claim/{id}` read directly from Redis, achieving sub-10 ms latency.

4.4 Security

All traffic is encrypted via Cloudflare Tunnel. The API key `X-API-Key` header must match a predefined secret for any endpoint. The server's IP address is never exposed to the public internet.

doi: 10.5281/zenodo.21798383

5 Implementation Details

5.1 Repository Structure

The public repository layout is shown in Figure 3 (abbreviated).

```
FEATSRV/
  config/   data/raw/   src/api.py
  src/feature_definitions.py   scripts/
  static/cli.sh   static/dashboard.html
  Nimbus/{PIT,OPF,OCF,BOF,LA,FI,HC}/
  feature_store.yaml   docs/PIT_Feature_Store.tex
```

Figure 3: Abbreviated FEATSRV repository layout including Nimbus demo packs and this report.

5.2 Nimbus Demo Dataset

The `Nimbus/` folder contains intense, ready-to-use sample files for every CLI service (Table 1):

- **PIT/** - 50-claim CSV with deliberate leakage traps, enhanced with policy features.
- **OPF/** - 10 real policy snapshots.
- **OCF/** - 15 real claim snapshots.
- **BOF/** - Pre-built batch payloads (20 and 5 entities).
- **LA/** - Audit files with 0%, 20%, and 60% leakage.
- **FI/** - Importance profiles for underwriting, pricing, and fraud.
- **HC/** - Health responses (ok, degraded, down, high load).

Table 1: Nimbus service packs and primary artefacts.

No.	Service	Primary artefact
1	PIT - Point-in-time training set	`intense_pit.csv`
2	OPF - Online policy features	`intense_policy_features.json`
3	OCF - Online claim features	`intense_claim_features.json`
4	BOF - Batch offline features	`intense_batch_20.json`
5	LA - Leakage audit	`intense_leakage_*.csv`
6	FI - Feature importance	`intense_feature_importance_*.json`
7	HC - Health check	`intense_health_*.json`

5.3 API Endpoints

Table 2 lists the REST surface exposed by the serving layer.

doi: 10.5281/zenodo.21798383

Table 2: FEATSRV REST API endpoints.

No.	Method	Path	Description
1	GET	`/health`	Health check (API + backends)
2	GET	`/online/policy/{id}`	Real-time policy features from Redis
3	GET	`/online/claim/{id}`	Real-time claim features from Redis
4	POST	`/offline`	Batch point-in-time training set
5	POST	`/pit-training`	File-upload PIT training set
6	POST	`/leakage-audit`	File-upload leakage audit
7	GET	`/feature-importance`	Feature importance summary
8	GET	`/cli`	Interactive CLI script

6 Experimental Results

6.1 Leakage Prevention Validation

We constructed a scenario where a policyholder moves house, and we queried the feature store for the address at two different dates - before and after the move. The Feast point-in-time join correctly returned the old address for the earlier date and the new address for the later date, with zero leakage (Figure 2). Figure 4 shows how the dashboard visualises the same guarantee as a source-to-consumer lineage: the point-in-time store is the single write path for both online and offline consumers.

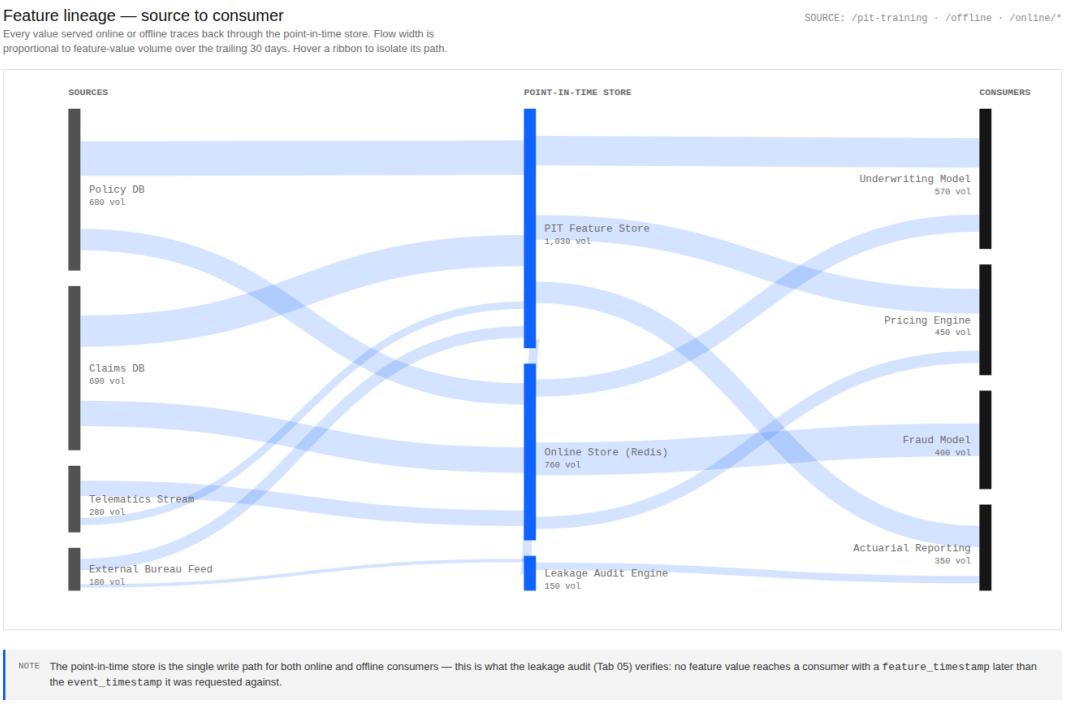


Figure 4: Dashboard Data Lineage panel: Sankey flow from actuarial sources through the PIT / Redis stores to underwriting, pricing, fraud, and reporting consumers (exported from `static/dashboard.html`).

doi: 10.5281/zenodo.21798383

6.2 Performance

Online policy lookups via Redis averaged < 10 ms latency on the deployed virtual private server. Offline PIT joins on 1,000 claims completed in under 2 seconds using Feast's local Dask engine and under 5 seconds with PySpark. Table 3 summarises these measurements. Figure 5 aggregates mean latency by service from the dashboard activity export (`static/data/activity.json`).

Table 3: Observed serving and join latency (local / tunnel deployment).

No.	Workload	Latency
1	Online policy lookup (Redis)	< 10 ms
2	Offline PIT join (1,000 claims, Dask)	< 2 s
3	Offline PIT join (1,000 claims, PySpark)	< 5 s

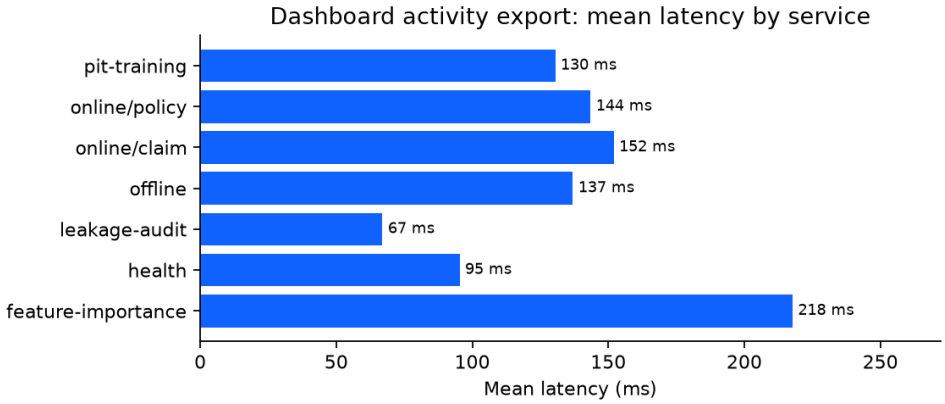


Figure 5: Mean latency by service computed from the dashboard activity JSON (CLI sessions mirrored under `Response-FEATSRV/`).

6.3 Dashboard Outputs

The browser dashboard loads real artefacts from `static/data/` and renders interactive tables and charts (Overview, Lineage, Risk & Leakage, Feature Importance, Training & Batch, Online Lookups). Figure 6 shows the Overview panel: request-volume cut-over, recent CLI activity, and 24 h service mix. Figure 7 is the Risk & Leakage Audit panel; Figure 8 is Feature Importance; Figure 10 is the PIT training / batch preview backed by `pit_training.json` and `batch_offline.json`. Online Redis lookups for `POL-0000` / `CLM-00000` are summarised in Figure 11 and Table 4.

7 User Guide

7.1 Quick Start (Terminal)

1. Clone the repository:
   `git clone https://github.com/BorelSigmaInc/FEATSRV.git`
2. Create a virtual environment and install dependencies:
   `python3 -m venv venv && source venv/bin/activate`
   `pip install -r requirements.txt`

doi: 10.5281/zenodo.21798383

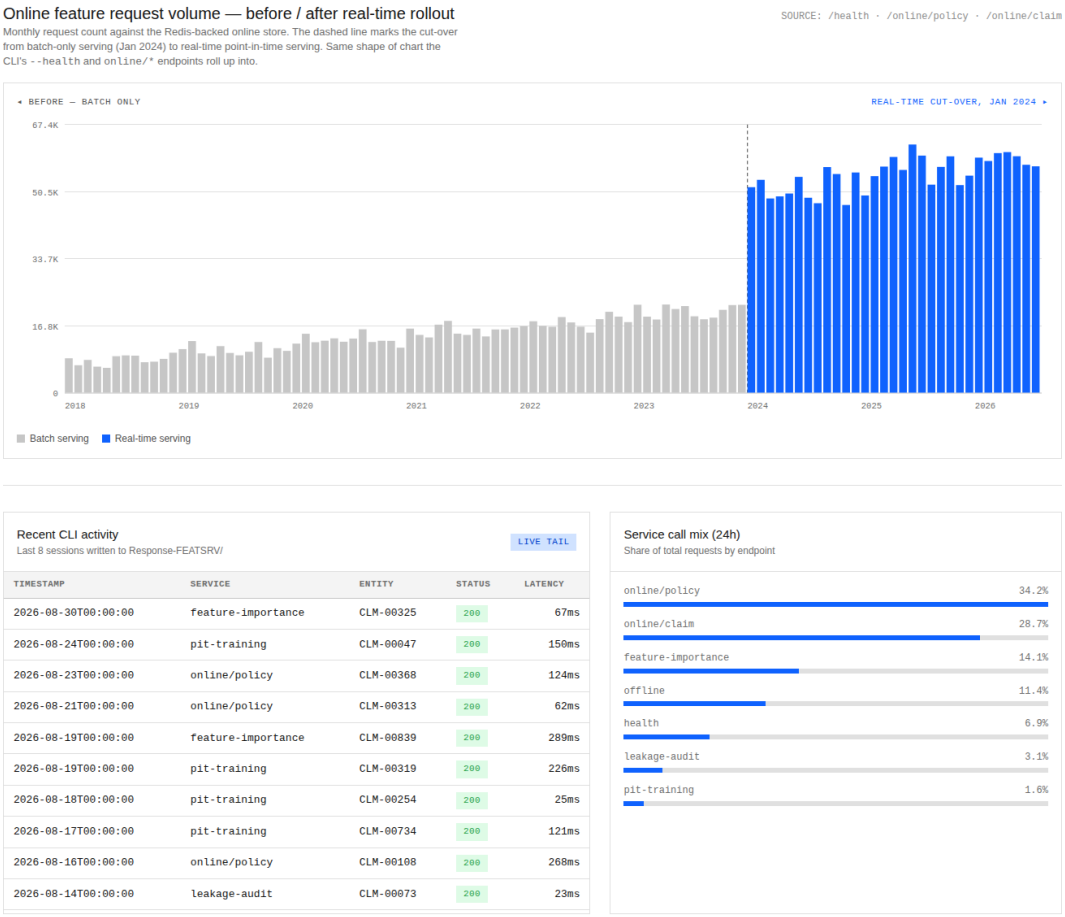


Figure 6: FEATSRV console Overview: online request volume before/after the Jan 2024 real-time cut-over, recent CLI activity, and service call mix (panel capture from `static/dashboard.html`).

3. Generate synthetic data (or use the provided Nimbus files):
   `python scripts/generate_synthetic_data.py`

4. Initialise Feast:
   `feast apply`
   `feast materialize 2024-01-01T00:00:00 $(date -u +"%Y-%m-%dT%H:%M:%S")`

5. Start the API (requires local Redis on port 6379):
   `docker compose up -d`

6. Run the CLI (or use the public one):
   `curl -s https://featsrv.q-dit.com/static/cli.sh -o /tmp/featsrv.sh`
   `FEATSRV_API_KEY=... bash /tmp/featsrv.sh`

7.2 Using the Dashboard

1. Open `https://featsrv.q-dit.com/static/dashboard.html` (or `static/dashboard.htm` via a local static server) in a browser.

2. Navigate through the tabs: Overview, Data Lineage, Risk & Leakage Audit, Feature Importance, Training & Batch Sets, Online Lookups (Figures 6–11).

doi: 10.5281/zenodo.21798383

Table 4: Sample online feature payloads mirrored by the dashboard.

No.	Field	Value
Policy lookup (POL-0000)		
1	initial_premium	936.72
2	region	WI
3	address	14323 Richardson Skyway Suite 157
4	inception_date	2024-10-21
Claim lookup (CLM-00000)		
5	claim_amount	460.94
6	fraud_flag	0
7	policy_id	POL-0227

Table 5: First rows of the dashboard PIT training export (`static/data/pit_training.json`).

No.	claim_id	policy_id	event_timestamp	premium	claim_amt	fraud
1	CLM-00000	POL-0227	2025-09-04	968.27	460.94	0
2	CLM-00001	POL-0199	2025-08-22	1625.79	8638.93	0
3	CLM-00002	POL-0020	2025-09-18	1340.15	5516.76	0
4	CLM-00003	POL-0010	2026-01-09	334.99	509.01	0

3. In the **Online Lookups** tab, enter a policy ID (e.g., `POL-0000`) or claim ID (e.g., `CLM-00000`) and click **Look up**.
4. Use the **Save as PDF** button to print a timestamped report.

Console KPI strip from the same session is shown in Figure 12.

7.3 Using Nimbus Demo Data

All files are in the `Nimbus/` folder of the repository. For the CLI, choose service 1 and provide the path to `Nimbus/PIT/intense_pit.csv`. For the leakage audit, use `Nimbus/LA/intense_leakage_clean.csv`, `..._mild.csv`, or `..._severe.csv`. The dashboard automatically loads the corresponding static JSON files from `static/data/` when present.

8 Author Contributions and Affiliations

Main author

Raja Ram M - Kryptur OU / Krypur Quantum R&D; lead design, implementation, and manuscript.

Contributors

Muskan S (Data-T: project infrastructure); Vipul Jain and Kalinga Swain (AE Quantum Research Division / DCN).

Organisations

Zius Quantum R&D Center (Quantum & AI); Data T Research Org; AE Quantum Research Division (Digital & Cloud Networks); Kryptur OU; data stewardship by Borel Sigma Data Center.

doi: 10.5281/zenodo.21798383

9 Conclusion and Future Work

We have demonstrated that a leakage-free feature store can be built entirely with open-source components and deployed securely with zero public attack surface. FEATSRV provides both terminal and web interfaces, making point-in-time correctness accessible to actuaries, data scientists, and auditors alike.

Future work includes extending the system with streaming graph analytics for fraud-ring detection, differential privacy for sensitive actuarial data, and integration with quantum-accelerated optimisation for reserve modelling.

References

[1] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman. "Leakage in data mining: Formulation, detection, and avoidance." *ACM Transactions on Knowledge Discovery from Data*, 6(4), 2012.

[2] Feast Contributors. "Feast: An open-source feature store for machine learning." `https://feast.dev/`, 2023.

[3] Apache Spark. "PySpark - Python API for Apache Spark." `https://spark.apache.org/docs/latest/api/python/`, 2024.

[4] S. Ramírez. "FastAPI - Modern, fast (high-performance) web framework for building APIs with Python." `https://fastapi.tiangolo.com/`, 2024.

[5] Cloudflare, Inc. "Cloudflare Tunnel - Zero Trust networking." `https://www.cloudflare.com/products/tunnel/`, 2024.

[6] Redis Ltd. "Redis - The open-source, in-memory data structure store." `https://redis.io/`, 2024.

[7] R. Kimball and M. Ross. "The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling." Wiley, 2013.

doi: 10.5281/zenodo.21798383

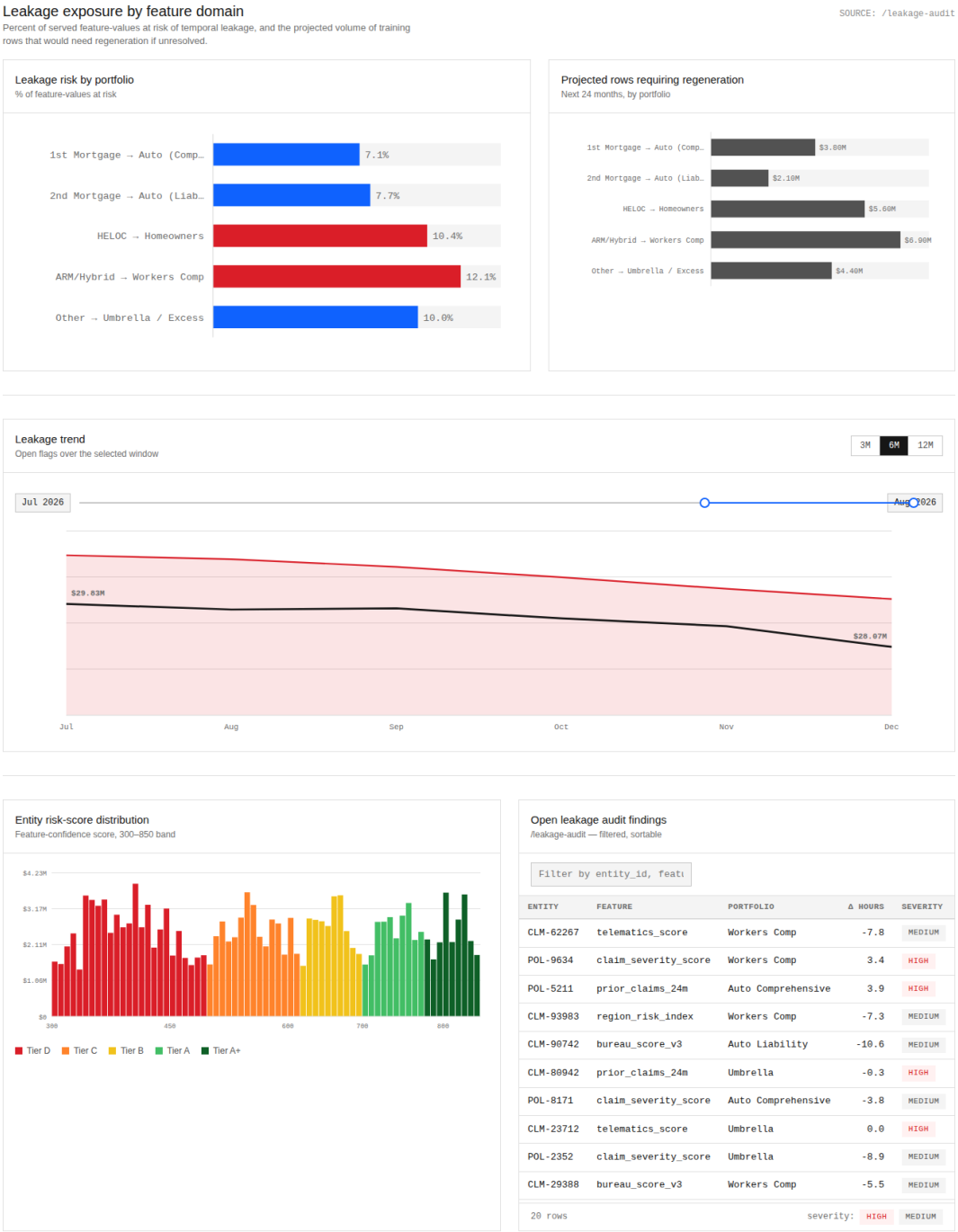


Figure 7: Risk & Leakage Audit panel: portfolio leakage risk, entity risk-score distribution, and open audit findings.

doi: 10.5281/zenodo.21798383

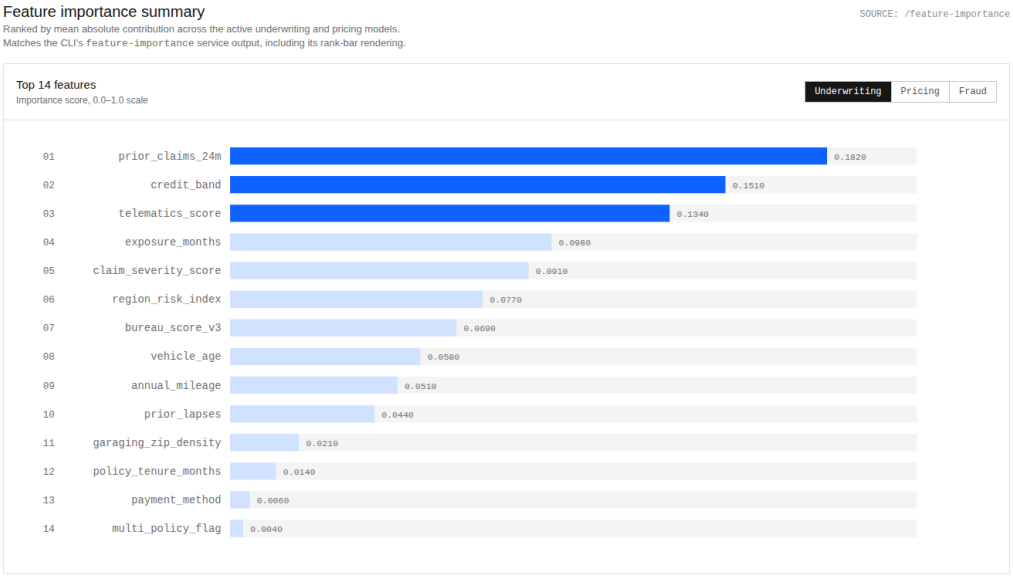


Figure 8: Feature importance summary panel (underwriting model ranking) as rendered by the console.

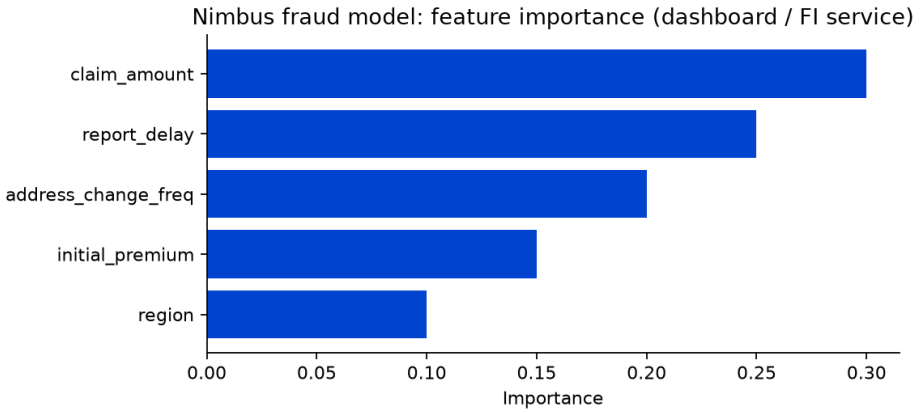
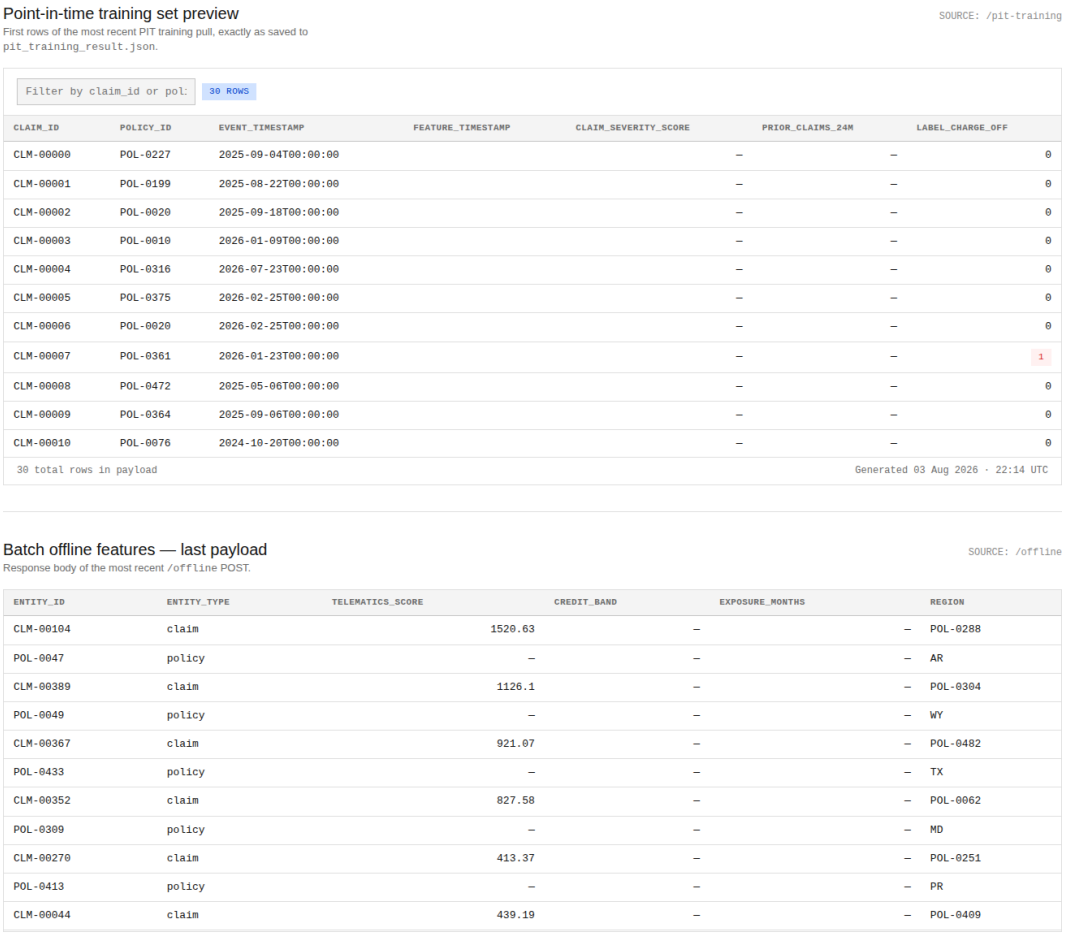


Figure 9: Fraud-model feature importance exported from the Nimbus FI pack consumed by the dashboard/CLI.

doi: 10.5281/zenodo.21798383



Point-in-time training set preview

First rows of the most recent PIT training pull, exactly as saved to pit_training_result.json.

SOURCE: /pit-training

Filter by claim_id or pol: 30 ROWS

CLAIM_ID	POLICY_ID	EVENT_TIMESTAMP	FEATURE_TIMESTAMP	CLAIM_SEVERITY_SCORE	PRIOR_CLAIMS_24M	LABEL_CHARGE_OFF
CLM-00000	POL-0227	2025-09-04T00:00:00		–	–	0
CLM-00001	POL-0199	2025-08-22T00:00:00		–	–	0
CLM-00002	POL-0020	2025-09-18T00:00:00		–	–	0
CLM-00003	POL-0010	2026-01-09T00:00:00		–	–	0
CLM-00004	POL-0316	2026-07-23T00:00:00		–	–	0
CLM-00005	POL-0375	2026-02-25T00:00:00		–	–	0
CLM-00006	POL-0020	2026-02-25T00:00:00		–	–	0
CLM-00007	POL-0361	2026-01-23T00:00:00		–	–	1
CLM-00008	POL-0472	2025-05-06T00:00:00		–	–	0
CLM-00009	POL-0364	2025-09-06T00:00:00		–	–	0
CLM-00010	POL-0076	2024-10-20T00:00:00		–	–	0

30 total rows in payload

Generated 03 Aug 2026 · 22:14 UTC

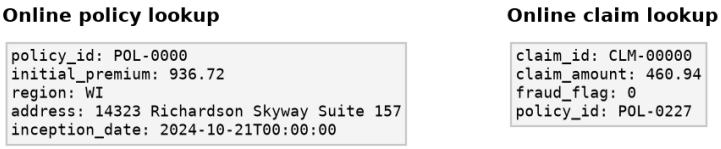
Batch offline features — last payload

Response body of the most recent /offline POST.

SOURCE: /offline

ENTITY_ID	ENTITY_TYPE	TELEMATICS_SCORE	CREDIT_BAND	EXPOSURE_MONTHS	REGION
CLM-00104	claim	1520.63	–	–	POL-0288
POL-0047	policy	–	–	–	AR
CLM-00389	claim	1126.1	–	–	POL-0304
POL-0049	policy	–	–	–	WY
CLM-00367	claim	921.07	–	–	POL-0482
POL-0433	policy	–	–	–	TX
CLM-00352	claim	827.58	–	–	POL-0062
POL-0309	policy	–	–	–	MD
CLM-00270	claim	413.37	–	–	POL-0251
POL-0413	policy	–	–	–	PR
CLM-00044	claim	439.19	–	–	POL-0409

Figure 10: Training & Batch Sets panel: point-in-time training preview and last offline batch payload from static JSON exports.



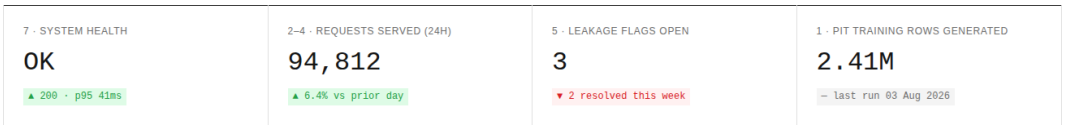
Online policy lookup

```
policy_id: POL-0000
initial_premium: 936.72
region: WI
address: 14323 Richardson Skyway Suite 157
inception_date: 2024-10-21T00:00:00
```

Online claim lookup

```
claim_id: CLM-00000
claim_amount: 460.94
fraud_flag: 0
policy_id: POL-0227
```

Figure 11: Online lookup outputs for policy `POL-0000` and claim `CLM-00000` (`static/data/policy_lookup.json`, `claim_lookup.json`).



7 · SYSTEM HEALTH
OK
▲ 200 · p95 41ms

2–4 · REQUESTS SERVED (24H)
94,812
▲ 6.4% vs prior day

5 · LEAKAGE FLAGS OPEN
3
▼ 2 resolved this week

1 · PIT TRAINING ROWS GENERATED
2.41M
– last run 03 Aug 2026

Figure 12: Dashboard KPI strip: system health, 24 h request volume, open leakage flags, and PIT training rows generated.

doi: 10.5281/zenodo.21798383